

SOFTWARE REQUIREMENTS SPECIFICATION

Vehicle Rental System

A Browser-Based Vehicle Booking Application with Client-Side Persistence

Project Type: Frontend Web Development Project **Technology:** HTML5, CSS3, JavaScript (ES6), Browser Local Storage **Course:** B.Tech, Semester III **Prepared by:** Naman Agarwal **Roll Number:** 22515000081 **Project Guide:** Gautam Mukherjee **Institution:** GLA University, Greater Noida
Document Version: 1.0 **Date:** 19 August 2026 **Document Status:** Baseline

Revision History

Version	Date	Author	Description of Change
0.1	12 August 2026	Naman Agarwal	Initial draft — project idea, objectives and module list derived from the approved synopsis
1.0	19 August 2026	Naman Agarwal	Baseline release. Complete data model, business rules, functional and non-functional requirements, test plan, limitations and future enhancements

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. Data Requirements
- 4. Business Rules
- 5. Functional Requirements
- 6. External Interface Requirements
- 7. Non-Functional Requirements
- 8. System Models
- 9. Verification and Testing
- 10. Limitations of the Present Scope
- 11. Future Enhancements
- 12. Conclusion
- 13. Appendices

1. Introduction

1.1 Purpose

This document specifies the software requirements for the **Vehicle Rental System**, a browser-based application that allows a user to browse a catalogue of rental vehicles, search and filter that catalogue, view the specifications of an individual vehicle, calculate the rental amount for a chosen date range, and record a booking.

The document is intended to serve as the agreed statement of what the system shall do. It defines the scope of the work, the data the system manages, the rules that govern its behaviour, and the criteria against which the finished application will be evaluated. It does not prescribe implementation details beyond the technology constraints stated in Section 2.5.

1.2 Document Conventions

The following conventions are used throughout this document.

Convention	Meaning
shall	A mandatory requirement. The system is incomplete without it.
should	A recommended behaviour. Its absence does not fail the requirement.
may	An optional behaviour left to the developer's discretion.
<code>monospace</code>	A literal identifier, key name, file name or code element.
FR-x.y	Functional requirement identifier, where x is the module number.
BR-nn	Business rule identifier.
NFR-nn	Non-functional requirement identifier.
TC-nn	Test case identifier.

Requirement identifiers are stable. If a requirement is withdrawn in a later revision, its identifier is retired rather than reused.

1.3 Intended Audience

Reader	Reason for reading
Project Guide	To evaluate the completeness and correctness of the proposed system.
Examiner / Evaluator	To verify that the delivered application satisfies the stated requirements.
Developer	To implement the system without ambiguity about expected behaviour.
Future maintainer	To understand the data model and rules before extending the system.

Readers primarily interested in *what the system does* should read Sections 2, 4 and 5. Readers interested in *how it will be judged complete* should read Sections 7 and 9.

1.4 Product Scope

The Vehicle Rental System is a multi-page web application that runs entirely inside a web browser. It has no server component, no database server and no network dependency after the page has loaded.

Within scope:

- A catalogue of rental vehicles (cars, bikes and SUVs) authored as static application data.
- Registration and login of users, with credentials held in browser storage.
- Keyword search and attribute-based filtering over the catalogue.
- A detail view showing the full specification and daily tariff of a selected vehicle.
- Automatic calculation of the rental amount from the pickup and return dates.
- Recording of confirmed bookings, and a history view listing them.
- A wishlist of vehicles saved by the user for later reference.
- A contact form with client-side validation.
- A responsive layout usable on mobile, tablet and desktop screen widths.

Outside scope:

- Any server-side processing, REST API or hosted database.
- Real payment collection or integration with a payment gateway.
- Genuine, secure user authentication (see Section 10.1).
- Live vehicle availability shared between different users or different devices.
- Administrative management of the catalogue through the user interface.
- Email or SMS delivery of any kind.

The wider business benefit stated in the synopsis — removing the need for a customer to visit a rental office — is demonstrated by this system rather than fully delivered by it. A production deployment would additionally require the components listed in Section 11.

1.5 Definitions, Acronyms and Abbreviations

Term	Definition
Catalogue	The fixed set of vehicles available for rent, defined in application data.
Booking	A record that a user has reserved a specific vehicle for a specific date range.
Rental period	The number of chargeable days between the pickup date and the return date.
Tariff	The rental charge per day for a vehicle, expressed in Indian Rupees.
Wishlist	A per-user list of vehicles saved for later viewing.
Session	The record of which user is currently logged in on this browser.
Local Storage	The browser API (<code>window.localStorage</code>) that stores string key-value pairs persistently on the user's device.
DOM	Document Object Model — the browser's in-memory representation of a page.
ES6	ECMAScript 2015, the JavaScript language version used in this project.
SRS	Software Requirements Specification — this document.
UI	User Interface.

1.6 References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*. Used as the structural basis of this document, adapted to the size of an academic project.
2. MDN Web Docs, *Web Storage API*. Reference for the persistence mechanism described in Section 3.5.
3. Project Synopsis, *Vehicle Rental System*, submitted by Naman Agarwal. Source of the module list in Section 2.2.

2. Overall Description

2.1 Product Perspective

The Vehicle Rental System is a new, self-contained product. It is not a replacement for, or a component of, any existing system.

Its architecture is deliberately flat. There are three layers, all executing in the browser:

Layer	Responsibility	Implemented in
Presentation	Page structure and visual styling	HTML5, CSS3
Application logic	Validation, search, filtering, cost calculation, page behaviour	JavaScript (ES6)
Persistence	Storage of users, bookings, wishlist entries and messages	Browser Local Storage

The vehicle catalogue is not part of the persistence layer. It is authored once as a JavaScript data structure and treated as read-only application data, because in the present scope no user is permitted to add or alter a vehicle.

A consequence of this architecture, which is important to state plainly, is that **all data is local to one browser on one device**. Two users of the same application on two different machines do not share any state.

2.2 Product Functions

The system is organised into nine functional modules, matching the module list in the approved synopsis.

#	Module	Summary of function
1	Home	Landing page. Introduces the service and provides entry points to the catalogue.
2	Login and Signup	Registration of a new user and login of an existing one.

#	Module	Summary of function
3	Vehicle Listing	Displays the catalogue as a grid of vehicle cards.
4	Search and Filter	Narrows the listing by keyword and by vehicle attributes.
5	Vehicle Details	Displays the full specification and tariff of one vehicle.
6	Booking System	Accepts a date range, computes the rental amount, and records the booking.
7	Wishlist	Saves and removes vehicles against the logged-in user.
8	Booking History	Lists the bookings previously made by the logged-in user.
9	Contact	Accepts a message through a validated form.

2.3 User Classes and Characteristics

The system has a single user class in the present scope.

User class	Characteristics	Technical skill assumed
Customer	Wishes to browse vehicles and record a rental booking. Uses either a mobile phone or a laptop.	Able to use a website and fill a form. No technical knowledge assumed.

A **guest** is a visitor who has not logged in. A guest is not a separate user class but a state of the same class: guests may browse, search, filter and view details, but may not book, use the wishlist, or view booking history (BR-02).

An administrator class is deliberately absent. The catalogue is fixed at development time, so no administrative function exists to be specified.

2.4 Operating Environment

Item	Requirement
Device	Any desktop, laptop, tablet or smartphone.
Operating system	Any, provided a supported browser is available.
Browser	A current version of Google Chrome, Microsoft Edge, Mozilla Firefox or Safari, with JavaScript enabled and Local Storage permitted.
Network	Required only to load the application files. No network access is needed thereafter.
Server	None. The application runs from the local file system or from any static web host.
Development environment	Visual Studio Code.

2.5 Design and Implementation Constraints

These constraints arise from the academic requirements of the project and are not negotiable within the present scope.

ID	Constraint
C-01	The application shall be built using HTML5, CSS3 and JavaScript (ES6) only.
C-02	No frontend framework or library (React, Angular, Vue, jQuery, Bootstrap or similar) shall be used.
C-03	No backend server, server-side language or database server shall be used.
C-04	All persistent data shall be held in browser Local Storage.
C-05	The application shall function when opened directly from the file system, without a web server.
C-06	All vehicle images shall be stored within the project folder. No external image hosting shall be relied upon.

Constraint C-05 has a direct design consequence: because a page opened over the `file://` protocol cannot reliably fetch a separate JSON file, the vehicle catalogue shall be declared as a JavaScript data structure in a script file rather than loaded asynchronously.

2.6 Assumptions and Dependencies

ID	Assumption
A-01	The user's browser has Local Storage enabled and is not in a private or incognito session, in which stored data would be discarded when the session ends.
A-02	The Local Storage quota available to the origin (typically several megabytes) is not exhausted by the volume of data this application writes.
A-03	The system clock of the user's device is set correctly, since date validation in BR-04 depends on the current date.
A-04	The vehicle catalogue, its images and its tariffs are fictional data authored for demonstration and are not represented as those of any real rental business.
A-05	Only one person uses a given browser profile. The system cannot distinguish two people sharing one device beyond the credentials they enter.

3. Data Requirements

3.1 Overview of Entities

Five entities are managed by the system.

Entity	Origin	Mutable at run time
Vehicle	Authored in application data	No
User	Created by registration	No (created only)
Booking	Created by the booking module	Created only
Wishlist entry	Created and removed by the wishlist module	Created and deleted
Contact message	Created by the contact module	Created only

3.2 Vehicle

A vehicle is a single rentable unit in the catalogue.

Field	Type	Description	Constraint
vehicleId	String	Unique identifier, e.g. V001	Mandatory, unique in the catalogue
name	String	Model name, e.g. Swift VXi	Mandatory
brand	String	Manufacturer, e.g. Maruti Suzuki	Mandatory
type	String	One of Car , Bike , SUV	Mandatory
fuelType	String	One of Petrol , Diesel , Electric	Mandatory
transmission	String	One of Manual , Automatic	Mandatory for Car and SUV ; Manual for Bike
seatingCapacity	Integer	Number of seats	Mandatory, greater than zero
pricePerDay	Integer	Tariff in whole Rupees per day	Mandatory, greater than zero
city	String	City where the vehicle is offered	Mandatory
imagePath	String	Relative path to the vehicle photograph	Mandatory
description	String	Two to three sentences of prose	Mandatory

Note on tariff type. pricePerDay is stored as a whole number of Rupees, not as a decimal. Because the rental amount is a whole-Rupee tariff multiplied by a whole number of days (BR-05), every monetary value in the system is an integer, and no floating-point rounding error can arise. This is a deliberate design decision, not an incidental one.

3.3 User

Field	Type	Description	Constraint
userId	String	Unique identifier generated at registration	Mandatory, unique
fullName	String	The user's name as entered	Mandatory, minimum 3 characters
email	String	Login identifier	Mandatory, unique, must satisfy BR-01
password	String	Login secret	Mandatory, minimum 6 characters
registeredAt	String	ISO-8601 timestamp of registration	Set by the system

The `password` field is stored in Local Storage in readable form. This is an unavoidable consequence of constraint C-03 and is recorded as a limitation in Section 10.1, not presented as a security feature.

3.4 Booking, Wishlist Entry and Contact Message

Booking

Field	Type	Description	Constraint
bookingId	String	Unique identifier, e.g. BK1723987654321	Mandatory, unique
userId	String	The user who made the booking	Mandatory, must match an existing user
vehicleId	String	The vehicle booked	Mandatory, must match a catalogue vehicle
pickupDate	String	Start of the rental, YYYY-MM-DD	Mandatory, satisfies BR-04
returnDate	String	End of the rental, YYYY-MM-DD	Mandatory, satisfies BR-04
days	Integer	Chargeable days, computed by BR-05	Mandatory, minimum 1
pricePerDay	Integer	Tariff at the time of booking	Copied from the vehicle
totalAmount	Integer	$\text{days} \times \text{pricePerDay}$	Mandatory
bookedAt	String	ISO-8601 timestamp of confirmation	Set by the system

The tariff is copied into the booking rather than referenced. A booking record therefore remains an accurate account of what was agreed even if the catalogue tariff is later edited by the developer.

Wishlist entry

Field	Type	Description	Constraint
userId	String	Owner of the entry	Mandatory
vehicleId	String	Vehicle saved	Mandatory
addedAt	String	ISO-8601 timestamp	Set by the system

The pair (`userId`, `vehicleId`) is unique, as required by BR-07.

Contact message

Field	Type	Description	Constraint
messageId	String	Unique identifier	Mandatory, unique
name	String	Sender's name	Mandatory, minimum 3 characters
email	String	Sender's email address	Mandatory, satisfies BR-01
subject	String	Short subject line	Mandatory
message	String	Body of the message	Mandatory, minimum 10 characters
submittedAt	String	ISO-8601 timestamp	Set by the system

3.5 Persistence Scheme

All persistent collections are serialised to JSON and written to Local Storage under the following keys.

Key	Contents
<code>vrs.users</code>	Array of User records
<code>vrs.session</code>	The <code>userId</code> of the user currently logged in, or absent if none
<code>vrs.bookings</code>	Array of Booking records for all users of this browser
<code>vrs.wishlist</code>	Array of Wishlist entries for all users of this browser
<code>vrs.messages</code>	Array of Contact message records

The `vrs.` prefix prevents collision with any other application served from the same origin.

The vehicle catalogue is **not** stored in Local Storage. It is declared in a script file and read directly, since it is immutable at run time (Section 3.1).

3.6 Identifier Generation

Entity	Generation rule
Vehicle	Assigned manually at authoring time, sequentially from <code>V001</code> .
User	U followed by the current epoch millisecond value.
Booking	BK followed by the current epoch millisecond value.
Contact message	MSG followed by the current epoch millisecond value.

Epoch-based identifiers are sufficient here because records are created one at a time by a single user in a single browser tab, so two identifiers cannot be generated in the same millisecond.

4. Business Rules

These rules govern system behaviour independently of any single screen. Functional requirements in Section 5 refer to them by identifier rather than restating them.

ID	Rule
BR-01	An email address is accepted only if it contains exactly one @ symbol, at least one character before it, and a dot-separated domain after it with at least two characters following the final dot.
BR-02	Booking, wishlist and booking history are available only to a logged-in user. A guest attempting these actions shall be directed to the login page.
BR-03	An email address may be registered only once. Registration with an email already present in <code>vrs.users</code> shall be rejected.
BR-04	The pickup date shall not be earlier than the current date. The return date shall not be earlier than the pickup date.
BR-05	The rental period in days is the difference between the return date and the pickup date, measured in whole calendar days. If both dates are the same day, the rental period is one day. The minimum chargeable period is therefore one day.
BR-06	The rental amount is $\text{days} \times \text{pricePerDay}$, in whole Rupees. No tax, deposit, discount or other adjustment is applied in the present scope.
BR-07	A vehicle may appear at most once in a given user's wishlist. Adding a vehicle already present shall have no effect other than an informative message.
BR-08	A vehicle shall not be bookable by the same user for a date range that overlaps an existing booking of that vehicle by that user. Two ranges overlap when the pickup date of one is not later than the return date of the other, and the return date of the first is not earlier than the pickup date of the other.
BR-09	Booking history and wishlist shall display only records whose <code>userId</code> matches the current session.
BR-10	Every form field marked mandatory in Section 3 shall be validated before the record is written. A record shall not be written when any validation fails.
BR-11	A booking, once written, shall not be modified or deleted in the present scope.

ID	Rule
BR-12	Logging out shall clear <code>vrs.session</code> only. Users, bookings, wishlist entries and messages shall remain in storage.

Note on BR-08. The overlap check compares a booking request only against bookings held in the same browser. It is a correctness rule for a single user's own history, not a guarantee of real-world fleet availability. Section 10.3 states this limitation.

5. Functional Requirements

5.1 Module 1 — Home

ID	Requirement
FR-1.1	The home page shall display the name of the service, a short introductory statement, and a primary link to the vehicle listing page.
FR-1.2	The home page shall display a navigation bar linking to Vehicle Listing, Wishlist, Booking History and Contact, present on every page of the application.
FR-1.3	The navigation bar shall display a Login link when no session exists, and the logged-in user's name together with a Logout control when a session exists.
FR-1.4	The home page should display a small selection of vehicles drawn from the catalogue as a preview, each linking to its detail page.
FR-1.5	On viewport widths below the mobile breakpoint defined in NFR-01, the navigation bar shall collapse into a toggle control that reveals the links when activated.

5.2 Module 2 — Login and Signup

ID	Requirement
FR-2.1	The signup form shall accept full name, email address, password and password confirmation.
FR-2.2	The system shall validate every signup field per BR-10, applying BR-01 to the email and the minimum lengths stated in Section 3.3.
FR-2.3	The system shall reject signup when the two password fields do not match, and shall report which field is at fault.
FR-2.4	The system shall reject signup when the email is already registered, per BR-03.
FR-2.5	On successful signup, the system shall append a User record to <code>vrs.users</code> , create a session per FR-2.8, and navigate to the vehicle listing page.
FR-2.6	The login form shall accept an email address and a password.
FR-2.7	The system shall accept a login only when a User record exists with a matching email and password. A failed login shall produce a single message that does not disclose which of the two values was wrong.
FR-2.8	On successful login, the system shall write the user's <code>userId</code> to <code>vrs.session</code> .
FR-2.9	The logout control shall clear <code>vrs.session</code> per BR-12 and return the user to the home page.
FR-2.10	Validation messages shall appear next to the field concerned, not as a browser alert dialog.

5.3 Module 3 — Vehicle Listing

ID	Requirement
FR-3.1	The listing page shall render the catalogue as a grid of cards, one card per vehicle.
FR-3.2	Each card shall display the vehicle image, name, brand, type, fuel type, seating capacity and tariff per day.
FR-3.3	Each card shall provide a control that opens the detail page for that vehicle.
FR-3.4	Each card shall provide a control that adds the vehicle to the wishlist, subject to BR-02 and BR-07.

ID	Requirement
FR-3.5	The grid shall reflow to fewer columns as viewport width decreases, per NFR-01.
FR-3.6	The page shall display the number of vehicles currently shown.
FR-3.7	When no vehicle satisfies the active search and filter criteria, the page shall display an explanatory message in place of the grid, together with a control that clears all criteria.

5.4 Module 4 — Search and Filter

ID	Requirement
FR-4.1	The listing page shall provide a text input that matches the entered text against the vehicle name and brand, ignoring case and leading or trailing spaces.
FR-4.2	Search results shall update as the user types, without requiring a separate submit action.
FR-4.3	The page shall provide a filter on vehicle type, offering Car , Bike , SUV and an "all types" option.
FR-4.4	The page shall provide a filter on fuel type, offering Petrol , Diesel , Electric and an "all fuels" option.
FR-4.5	The page shall provide a filter on transmission, offering Manual , Automatic and an "any" option.
FR-4.6	The page shall provide a maximum-tariff filter that excludes vehicles whose pricePerDay exceeds the selected value.
FR-4.7	Search text and all active filters shall combine conjunctively: a vehicle is shown only when it satisfies every active criterion.
FR-4.8	The page shall provide a sort control offering tariff ascending and tariff descending. Sorting shall apply to the filtered result, not to the whole catalogue.
FR-4.9	The page shall provide a control that clears the search text and resets every filter and the sort order to its default.

5.5 Module 5 — Vehicle Details

ID	Requirement
FR-5.1	The detail page shall identify the vehicle to display from a parameter carried in the page URL.
FR-5.2	When the identifier is absent or matches no catalogue vehicle, the page shall display a "vehicle not found" message and a link back to the listing page, and shall not display an empty template.
FR-5.3	The page shall display the image, name, brand, type, fuel type, transmission, seating capacity, city, description and tariff per day of the vehicle.
FR-5.4	The page shall display the booking form specified in Section 5.6.
FR-5.5	The page shall provide a control to add the vehicle to the wishlist, subject to BR-02 and BR-07.

5.6 Module 6 — Booking System

ID	Requirement
FR-6.1	The booking form shall accept a pickup date and a return date, entered through a date control.
FR-6.2	The system shall validate both dates against BR-04 and shall display the reason when validation fails.
FR-6.3	The date control shall set its earliest selectable value to the current date, so that a past pickup date cannot ordinarily be chosen. Validation under FR-6.2 shall be performed regardless, because the control's restriction can be bypassed.
FR-6.4	When both dates are valid, the system shall compute the rental period per BR-05 and the rental amount per BR-06, and display both together with the tariff used.
FR-6.5	The displayed amount shall update whenever either date changes, without requiring a separate action.
FR-6.6	The system shall reject a booking that overlaps an existing booking of the same vehicle by the same user, per BR-08, and shall display the dates of the conflicting booking.
FR-6.7	A guest attempting to confirm a booking shall be directed to the login page per BR-02.
FR-6.8	On confirmation, the system shall append a Booking record to <code>vrs.bookings</code> with every field specified in Section 3.4.

ID	Requirement
FR-6.9	Following a successful booking, the system shall display a confirmation containing the booking identifier, the vehicle name, both dates, the rental period and the total amount.
FR-6.10	The confirmation shall state explicitly that the booking is a simulation and that no payment has been taken.

5.7 Module 7 — Wishlist

ID	Requirement
FR-7.1	The wishlist page shall list the vehicles saved by the logged-in user, per BR-09.
FR-7.2	Each entry shall display the vehicle image, name, type and tariff, and shall link to the vehicle's detail page.
FR-7.3	Each entry shall provide a control that removes it from the wishlist.
FR-7.4	Adding a vehicle already in the wishlist shall be handled per BR-07.
FR-7.5	When the wishlist is empty, the page shall display a message to that effect and a link to the vehicle listing page.
FR-7.6	A guest opening the wishlist page shall be directed to the login page per BR-02.

5.8 Module 8 — Booking History

ID	Requirement
FR-8.1	The booking history page shall list the bookings made by the logged-in user, per BR-09.
FR-8.2	Each row shall display the booking identifier, vehicle name, pickup date, return date, rental period in days and total amount.
FR-8.3	Bookings shall be listed most recent first, ordered by <code>bookedAt</code> .
FR-8.4	The page shall display the total number of bookings and the sum of their amounts.
FR-8.5	When the user has no bookings, the page shall display a message to that effect and a link to the vehicle listing page.
FR-8.6	A guest opening the booking history page shall be directed to the login page per BR-02.
FR-8.7	The page shall not offer any control to edit or cancel a booking, per BR-11.

5.9 Module 9 — Contact

ID	Requirement
FR-9.1	The contact page shall provide a form accepting name, email address, subject and message.
FR-9.2	The system shall validate every field per BR-10, applying BR-01 to the email address.
FR-9.3	On successful validation, the system shall append a Contact message record to <code>vrs.messages</code> and clear the form.
FR-9.4	The page shall display a confirmation that states the message has been recorded locally and has not been transmitted to any recipient.
FR-9.5	The page should display static contact details — address, telephone number and email address — presented as illustrative data.

6. External Interface Requirements

6.1 User Interfaces

The application consists of the following pages.

Page	File	Purpose
Home	<code>index.html</code>	Landing page (Module 1)

Page	File	Purpose
Login	login.html	Login form (Module 2)
Signup	signup.html	Registration form (Module 2)
Vehicle Listing	vehicles.html	Catalogue, search and filters (Modules 3 and 4)
Vehicle Details	vehicle-details.html	Specification and booking form (Modules 5 and 6)
Wishlist	wishlist.html	Saved vehicles (Module 7)
Booking History	bookings.html	Past bookings (Module 8)
Contact	contact.html	Contact form (Module 9)

Interface requirements common to all pages:

ID	Requirement
UI-01	Every page shall carry the same navigation bar and footer.
UI-02	The navigation link corresponding to the current page shall be visually distinguished.
UI-03	Colour, typography and spacing shall be defined once in a shared stylesheet and reused, so that no page diverges in appearance.
UI-04	Every interactive control shall present a visible change of state on hover and on keyboard focus.
UI-05	Error messages shall be rendered inside the page, adjacent to the control concerned. Browser <code>alert</code> dialogs shall not be used for validation.
UI-06	Every image shall carry descriptive alternative text.

6.2 Hardware Interfaces

The application requires no hardware interface beyond the display, keyboard, pointer or touchscreen exposed to the browser. It does not use the camera, microphone, geolocation service or any other device capability.

6.3 Software Interfaces

Interface	Use
<code>window.localStorage</code>	Reading and writing the collections defined in Section 3.5.
<code>JSON.parse</code> / <code>JSON.stringify</code>	Serialisation of those collections.
<code>Date</code>	Current date for BR-04, and day arithmetic for BR-05.
DOM API	Rendering, event handling and form validation.

No third-party library, package manager, build step or external service is used.

6.4 Communications Interfaces

None. The application performs no network request after the page and its assets have loaded. It does not use `fetch`, `XMLHttpRequest`, `WebSockets` or any form of remote submission.

7. Non-Functional Requirements

7.1 Usability

ID	Requirement
NFR-01	The layout shall adapt at three breakpoints: a single-column layout below 576 px, a two-column layout from 576 px to 991 px, and a multi-column layout at 992 px and above.
NFR-02	No page shall require horizontal scrolling at any viewport width of 320 px or greater.

ID	Requirement
NFR-03	Any function shall be reachable from the home page within three clicks or taps.
NFR-04	Text shall maintain sufficient contrast against its background to remain legible, and body text shall not be rendered below 14 px.
NFR-05	Every form control shall have an associated visible label. Placeholder text shall not be used as the only label.

7.2 Reliability and Correctness

ID	Requirement
NFR-06	Invalid input shall never produce an unhandled JavaScript exception. Every validation failure shall be reported to the user.
NFR-07	Reading from Local Storage shall tolerate a missing or malformed value by falling back to an empty collection rather than failing to render the page.
NFR-08	Data written by one page shall be visible to every other page immediately on load, since all pages read the same keys.
NFR-09	Monetary values shall remain exact, as guaranteed by the integer arithmetic described in Sections 3.2 and 4 (BR-06).

7.3 Performance

ID	Requirement
NFR-10	Search and filter results shall be recomputed and re-rendered without perceptible delay for a catalogue of the size defined in Appendix A.
NFR-11	Filtering shall operate on the in-memory catalogue array. It shall not re-read Local Storage on each keystroke.
NFR-12	Total page assets excluding images shall remain small enough to load promptly on a mobile connection; images shall be compressed before inclusion in the project folder.

No absolute timing figures are stated. The catalogue size is fixed by the developer and every operation is a single pass over a small array, so a numeric threshold would be an arbitrary claim rather than a measured requirement.

7.4 Security

ID	Requirement
NFR-13	Text supplied by the user shall be inserted into the page as text content, not as markup, so that a value containing HTML is displayed literally rather than executed.
NFR-14	The application shall not transmit any user data to any external destination.
NFR-15	The signup form shall state that the password entered is stored locally in the browser, and shall advise the user not to reuse a password used elsewhere.

The system offers no confidentiality guarantee. Section 10.1 states this in full. NFR-13 to NFR-15 are the mitigations that are actually possible within constraint C-03; they are not represented as making the system secure.

7.5 Maintainability

ID	Requirement
NFR-16	JavaScript shall be organised into separate files by concern — catalogue data, storage helpers, authentication, listing and filtering, booking, wishlist — rather than held in one file.
NFR-17	Every read and write of Local Storage shall pass through a small set of shared helper functions, so that the key names in Section 3.5 appear in exactly one file.
NFR-18	Business rules BR-01, BR-04, BR-05 and BR-08 shall each be implemented as a single named function, called wherever the rule applies, so that no rule is expressed twice.

ID	Requirement
NFR-19	Code shall be commented where the intent is not evident from the code itself, particularly in the date arithmetic of BR-05 and the overlap test of BR-08.

7.6 Portability

ID	Requirement
NFR-20	The application shall run identically from the local file system and from a static web host, per constraint C-05.
NFR-21	All internal paths shall be relative. No absolute file-system path shall appear in any file.
NFR-22	The application shall behave consistently across the browsers listed in Section 2.4.

8. System Models

8.1 Principal User Flow — Making a Booking

Step	Actor	Action	Governing requirement
1	User	Opens the application at the home page	FR-1.1
2	User	Navigates to the vehicle listing	FR-1.2
3	User	Enters search text and applies filters	FR-4.1 to FR-4.7
4	System	Re-renders the grid with matching vehicles	FR-3.1, FR-4.2
5	User	Opens the detail page of a chosen vehicle	FR-3.3, FR-5.1
6	User	Selects pickup and return dates	FR-6.1
7	System	Validates the dates	FR-6.2, BR-04
8	System	Computes and displays the rental period and amount	FR-6.4, BR-05, BR-06
9	User	Confirms the booking	FR-6.8
10	System	Verifies a session exists	FR-6.7, BR-02
11	System	Verifies the dates do not overlap an existing booking	FR-6.6, BR-08
12	System	Writes the Booking record and shows the confirmation	FR-6.8, FR-6.9
13	User	Views the booking under Booking History	FR-8.1

Steps 10 and 11 are ordered deliberately. The session is checked first, because the overlap test in BR-08 is defined per user and cannot be evaluated without knowing which user is acting.

8.2 Module Dependency

Module	Depends on	Nature of dependency
Vehicle Listing	Catalogue data	Reads the vehicle array
Search and Filter	Vehicle Listing	Operates on the same in-memory array
Vehicle Details	Catalogue data	Looks up one vehicle by identifier
Booking System	Vehicle Details, Login	Needs the vehicle tariff and the session
Wishlist	Login, Catalogue data	Stores identifiers, resolves them for display
Booking History	Booking System, Login	Reads records the booking module wrote
Contact	None	Independent of every other module

The Contact module has no dependency and may therefore be implemented first as a self-contained exercise in form validation.

8.3 State of the Session

The application has exactly two session states.

State	Condition	Permitted actions
Guest	<code>vrs.session</code> absent	Browse, search, filter, view details, submit contact form
Authenticated	<code>vrs.session</code> holds a valid <code>userId</code>	All guest actions, plus book, wishlist and booking history

Transitions: Guest to Authenticated by successful login or signup (FR-2.5, FR-2.8); Authenticated to Guest by logout (FR-2.9). A `vrs.session` value that matches no user in `vrs.users` shall be treated as absent, per the tolerant reading required by NFR-07.

9. Verification and Testing

9.1 Approach

Testing is manual and exploratory. No automated test framework is used, since introducing one would conflict with constraint C-02 and lies outside the scope of the course work. Each test case below is executed in a browser and its result recorded before submission.

9.2 Test Cases

ID	Scenario	Expected result	Verifies
TC-01	Sign up with a valid, unused email	Account created, user logged in, listing page shown	FR-2.5
TC-02	Sign up with an email already registered	Rejected with an explanatory message	FR-2.4, BR-03
TC-03	Sign up with mismatched passwords	Rejected, the confirmation field identified	FR-2.3
TC-04	Sign up with <code>naman.example.com</code> as the email	Rejected as invalid	FR-2.2, BR-01
TC-05	Log in with a correct email and wrong password	Rejected with a message that does not identify which field was wrong	FR-2.7
TC-06	Search for a brand present in the catalogue	Only matching vehicles displayed	FR-4.1
TC-07	Search using different letter case and surrounding spaces	Same result as TC-06	FR-4.1
TC-08	Apply type <code>Bike</code> together with fuel <code>Electric</code>	Only vehicles satisfying both criteria displayed	FR-4.7
TC-09	Apply filters that match no vehicle	Empty-result message and a clear-criteria control displayed	FR-3.7
TC-10	Use the clear control after applying several filters	Full catalogue restored, all controls at default	FR-4.9
TC-11	Sort by tariff ascending after filtering	Filtered set ordered by tariff, unmatched vehicles still excluded	FR-4.8
TC-12	Open the detail page with an identifier that does not exist	"Vehicle not found" message displayed	FR-5.2
TC-13	Select a return date earlier than the pickup date	Rejected with the reason shown	FR-6.2, BR-04
TC-14	Select the same date for pickup and return	Rental period shown as 1 day, amount equal to one day's tariff	FR-6.4, BR-05
TC-15	Select a five-day range	Rental period shown as 5 days, amount equal to five times the tariff	FR-6.4, BR-06

ID	Scenario	Expected result	Verifies
TC-16	Change the return date after the amount is displayed	Amount recalculated without a page reload	FR-6.5
TC-17	Attempt to book while logged out	Redirected to the login page	FR-6.7, BR-02
TC-18	Book dates overlapping an existing booking of the same vehicle	Rejected, conflicting dates shown	FR-6.6, BR-08
TC-19	Complete a booking, then open Booking History	The booking appears with the correct amount	FR-8.1, FR-8.2
TC-20	Add the same vehicle to the wishlist twice	Single entry retained, informative message shown	FR-7.4, BR-07
TC-21	Log in as a second user	Neither the wishlist nor the history of the first user is visible	BR-09
TC-22	Log out, then reopen the application	Session cleared; users, bookings and wishlist entries retained	FR-2.9, BR-12
TC-23	Submit the contact form with a 4-character message	Rejected for insufficient length	FR-9.2
TC-24	Enter <code><script>alert(1)</script></code> as a full name and view it on a page	Text displayed literally, no script executed	NFR-13
TC-25	Reduce the viewport to 360 px and visit every page	No horizontal scrolling, navigation collapses correctly	NFR-01, NFR-02
TC-26	Manually corrupt a Local Storage value and reload	Page renders with an empty collection rather than failing	NFR-07

9.3 Acceptance Criteria

The system is accepted as complete when every functional requirement in Section 5 is implemented, every test case in Section 9.2 passes, and the application meets NFR-01, NFR-02 and NFR-06 on at least two of the browsers listed in Section 2.4.

10. Limitations of the Present Scope

These are consequences of the constraints in Section 2.5, stated here so that they are not mistaken for defects.

10.1 Authentication is a simulation, not a security mechanism

Passwords are stored in Local Storage in readable form (Section 3.3), because there is no server on which to verify a credential. Anyone with access to the browser's developer tools can read every stored account. The login module demonstrates the *interface* of authentication — form validation, session state, protection of pages by session — and should be understood in that light. NFR-15 requires the signup page to tell the user so.

10.2 Data is confined to one browser on one device

There is no shared store. A user who registers in Chrome cannot log in with the same account in Firefox, or on a second device. Clearing the browser's site data removes every account, booking, wishlist entry and message.

10.3 Availability is not genuine

The overlap test in BR-08 examines only the bookings held in the present browser. Two different people could book the same vehicle for the same dates without the system detecting a conflict, because neither browser can see the other's records. Genuine availability requires the shared database listed in Section 11.

10.4 The catalogue is fixed

Vehicles are authored at development time and cannot be added, edited or removed through the interface. There is no administrative role in the present scope (Section 2.3).

10.5 No payment is processed

The booking confirmation records an intent, not a transaction. FR-6.10 requires the confirmation to state this to the user.

10.6 Bookings cannot be cancelled or amended

BR-11 fixes a booking once written. Cancellation would require rules about refunds and about the release of dates back to availability, neither of which can be specified meaningfully without the shared store discussed in Section 10.3.

11. Future Enhancements

The following extensions are outside the present scope. They are listed in the order in which they would sensibly be undertaken, since each depends on the ones before it.

#	Enhancement	What it resolves
1	A backend service built with Node.js and Express	Provides a place for logic that cannot be trusted to the browser.
2	A database (MySQL or MongoDB) holding vehicles, users and bookings	Resolves Sections 10.2 and 10.4.
3	Server-side authentication with hashed passwords and signed session tokens	Resolves Section 10.1.
4	Availability checked centrally against all bookings	Resolves Section 10.3.
5	An administrative dashboard for managing the fleet and viewing bookings	Introduces the second user class deferred in Section 2.3.
6	Booking cancellation and amendment with a defined refund rule	Resolves Section 10.6.
7	Integration with a payment gateway	Resolves Section 10.5.
8	Email and SMS notification on booking confirmation	Requires items 1 and 2.
9	Map-based selection of pickup and drop-off points	An independent extension, requiring a mapping service.

Items 1 to 4 form the natural progression from this project into a full-stack one, and each of them is prevented today by a specific constraint in Section 2.5 rather than by a lack of effort.

12. Conclusion

This document specifies a vehicle rental application that is complete within a deliberately narrow boundary: one browser, no server, a fixed catalogue and simulated payment. Within that boundary the specification is intended to be exact — nine modules, a defined data model, twelve business rules that the functional requirements refer to rather than restate, and twenty-six test cases that verify them.

The value of the project lies in what it demonstrates rather than in what it deploys. Search and filtering exercise array processing and DOM rendering; the booking module exercises date arithmetic and validation; the wishlist and history modules exercise per-user data partitioning; and the responsive layout exercises CSS. Section 10 records honestly what the chosen architecture cannot do, and Section 11 maps each of those gaps onto the specific component that would close it.

13. Appendices

Appendix A — Catalogue Composition

The catalogue shall contain between 20 and 30 vehicles, distributed so that every filter in Section 5.4 returns a non-empty result for each of its options.

Type	Suggested count	Notes
Car	10 to 14	Mixed fuel types and both transmissions
Bike	5 to 8	Petrol and Electric; transmission Manual per Section 3.2
SUV	5 to 8	Mixed fuel types and both transmissions

At least two vehicles shall be present for each fuel type, and tariffs shall span a wide enough range for the maximum-tariff filter in FR-4.6 to be observably effective.

Appendix B — Suggested Project Structure

```
vehicle-rental-system/  
├── index.html  
├── login.html  
├── signup.html  
├── vehicles.html  
├── vehicle-details.html  
├── wishlist.html  
├── bookings.html  
├── contact.html  
├── css/  
│   ├── style.css  
│   └── responsive.css  
├── js/  
│   ├── vehicles-data.js  
│   ├── storage.js  
│   ├── auth.js  
│   ├── listing.js  
│   ├── booking.js  
│   ├── wishlist.js  
│   └── contact.js  
└── images/  
    └── vehicles/
```

This structure satisfies NFR-16 and NFR-17: `storage.js` is the only file that names a Local Storage key, and each remaining script file corresponds to one module of Section 5.

Appendix C — Requirement Index

Section	Identifier range	Count
2.5 Constraints	C-01 to C-06	6
2.6 Assumptions	A-01 to A-05	5
4 Business rules	BR-01 to BR-12	12
5 Functional requirements	FR-1.1 to FR-9.5	64
6.1 Interface requirements	UI-01 to UI-06	6
7 Non-functional requirements	NFR-01 to NFR-22	22
9.2 Test cases	TC-01 to TC-26	26

End of document.